

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Выпускающая кафедра: информационные технологии и автоматизированные системы (ИТАС)

ОТЧЕТ

«Оптимизация производственного заказа»

По дисциплине «Основы алгоритмизации и программирования»

Выполнил студент группы: ИВТ-25-16.

Лыгалов Андрей Ильич

Принял: Доц. Полякова О. А.

Пермь 2026

Содержание

Введение	3
1. Анализ предметной области	5
2. Выбор средств разработки	6
3. Проектирование программного обеспечения	7
4. Описание UML-диаграммы	9
5. Реализация программного средства	12
6. Основные задачи разработки и принятые решения.....	13
6.1 Организация файловой базы данных.....	13
6.2 Разделение интерфейса и бизнес-логики	13
6.3 Работа с вложенными компонентами	13
6.4 Проверка пользовательского ввода	13
6.5 Обработка нехватки времени.....	14
7. Тестирование программного средства	15
Заключение	16
Список использованных источников	17

Введение

В условиях современного производства важной задачей является своевременное обеспечение предприятия деталями, материалами икупаемыми позициями. При ручном планировании закупок легко ошибиться в сроках поставки, не учесть параллельность изготовления или забыть о вложенных компонентах изделия.

Разрабатываемая программа «Оптимизация производственного закупа» предназначена для расчета сроков, к которым детали и материалы должны быть на складе, а также для определения крайней и оптимальной даты закупки с учетом заданного запаса времени.

Целью работы является разработка настольного программного средства на языке C++ с использованием Qt Widgets, позволяющего вести базу изделий, деталей икупаемых материалов, а затем рассчитывать план производства и закупок по требуемой дате готовности.

Проект построен модульно: данные предметной области отделены от файловой базы, расчетный алгоритм вынесен в отдельный класс, а графический интерфейс отвечает за ввод, редактирование и отображение результатов.

Постановка задачи:

В ходе выполнения работы требовалось разработать программное средство, обеспечивающее выполнение следующих функций:

- создание записей об изделиях, деталях икупаемых материалах;
- указание единицы измерения, типа записи, срока изготовления или поставки;
- задание ограничения: сколько единиц можно изготовить параллельно или заказать одной партией;
- описание состава изделия через компоненты, включая вложенные компоненты;
- сохранение введенных данных в файловой базе для повторного использования;
- расчет дат, к которым компоненты должны находиться на складе;

- расчет крайнего срока закупки и оптимального срока закупки с запасом;
- вывод предупреждения, если по указанным срокам предприятие не успевает выполнить заказ;
- проверка корректности пользовательского ввода, включая даты и количества;
- создание простого и понятного русскоязычного интерфейса на Qt.

Программа должна учитывать не только верхний уровень изделия, но и всю структуру зависимостей. Например, если для стула нужна ножка, а для ножки нужен брус, то срок закупки бруса рассчитывается с учетом времени изготовления ножек и времени сборки самого стула.

1. Анализ предметной области

Производственный закуп связан с последовательностью зависимых операций. Готовое изделие может состоять из деталей, детали могут состоять из других материалов, а часть элементов может не изготавливаться, а закупаться у поставщика.

При планировании необходимо учитывать два разных типа записей: изготавливаемые позиции и закупаемые позиции. Для изготавливаемой позиции важны длительность производственного цикла и количество изделий, которое можно делать параллельно. Для закупаемой позиции важны срок поставки и ограничение одной партии закупки.

Основная сложность заключается в том, что сроки рассчитываются не вперед от даты обращения, а назад от даты, к которой должен быть готов итоговый продукт. Такой подход позволяет определить, когда каждый компонент должен быть на складе, и когда необходимо сделать заказ.

Для решения задачи используется иерархическая модель изделия. Каждая запись может иметь список компонентов, а каждый компонент ссылается на отдельную запись в базе. Это позволяет один раз занести материал или деталь, а затем повторно использовать ее в составе разных изделий.

2. Выбор средств разработки

В качестве основного языка программирования выбран C++. Он подходит для реализации настольных приложений, поддерживает объектно-ориентированное проектирование и позволяет удобно разделить программу на независимые модули.

Для графического интерфейса использовался фреймворк Qt 5.12.12 и модуль Qt Widgets. Такой выбор обусловлен наличием готовых элементов управления: окон, таблиц, деревьев, кнопок, полей ввода дат и чисел.

Редактирование и сборка проекта выполнялись в Qt Creator. Для проекта создан файл ProductionPurchaseOptimization.pro, позволяющий открыть и собрать приложение стандартными средствами Qt.

Для хранения пользовательских данных используется простая файловая база database.txt. Такой вариант не требует установки отдельной СУБД и подходит для нашего, учебного проекта, при этом оставляет возможность дальнейшего перехода на SQLite или другую базу данных.

Основные используемые технологии:

- C++17;
- Qt Widgets;
- Qt Creator;
- файловая база database.txt;
- UML-моделирование.

3. Проектирование программного обеспечения

При проектировании была выбрана модульная архитектура. Программа разделена на несколько частей: модели данных, база, расчетный модуль, функции работы с датами, формирование отчета и пользовательский интерфейс.

Модуль	Назначение	Основные данные
Models	Описывает предметную область	Date, Component, Item, ItemType
Database	Хранит и сохраняет базу изделий	ITEM и COMP в database.txt
Planner	Выполняет расчет сроков	PlanResult, StockNeed, PurchaseNeed
MainWindow	Реализует главное окно Qt	вкладки «База» и «Расчет»
ItemDialog	Создает и редактирует записи	дерево компонентов изделия

Основными структурами предметной области являются Date, Component и Item. Структура Date хранит дату без времени. Component описывает компонент, требуемый для одной единицы родительского изделия. Item описывает изделие, деталь или закупаемый материал.

Класс Database отвечает за загрузку, поиск, добавление, изменение, переименование и удаление записей. Данные сохраняются в текстовом файле, где строки ITEM описывают сами записи, а строки COMP описывают связи между родительским изделием и его компонентами.

Класс Planner реализует основную расчетную логику. Он получает ссылку на Database, находит нужное изделие, рекурсивно проходит по его составу и формирует результат PlanResult.

Графический интерфейс представлен классом MainWindow. Он содержит вкладку «База» для работы с деревом изделий и вкладку «Расчет» для выбора готового продукта, ввода количества, дат и запаса.

Диалог ItemDialog используется для создания и редактирования записи. В нем пользователь задает название, единицу измерения, тип записи, длительность цикла или поставки, ограничение параллельного изготовления или партии закупки, а также состав изделия.

4. Описание UML-диаграммы

Общая архитектура проекта

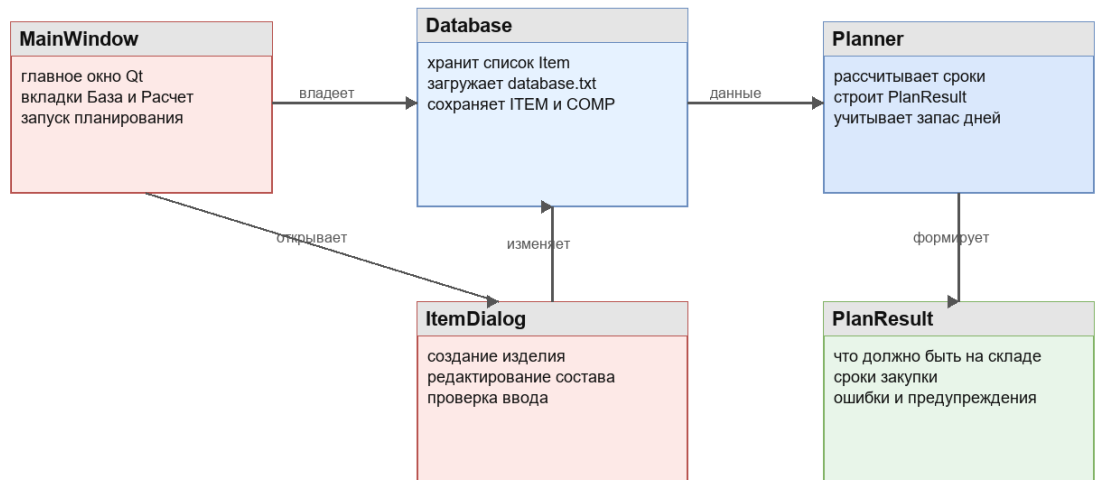


Рисунок 1 — Диаграмма классов. Общая архитектура

На первой диаграмме показана общая структура приложения. Центральными классами являются MainWindow, Database и Planner. MainWindow управляет интерфейсом и владеет объектом Database. При запуске расчета главное окно создает Planner, который читает данные из базы и формирует PlanResult.

ItemDialog связан с Database, так как при редактировании компонента пользователь может сразу создать или изменить вложенную запись. Это позволяет описывать многоуровневую структуру изделия без выхода из окна редактирования.

Структуры данных предметной области

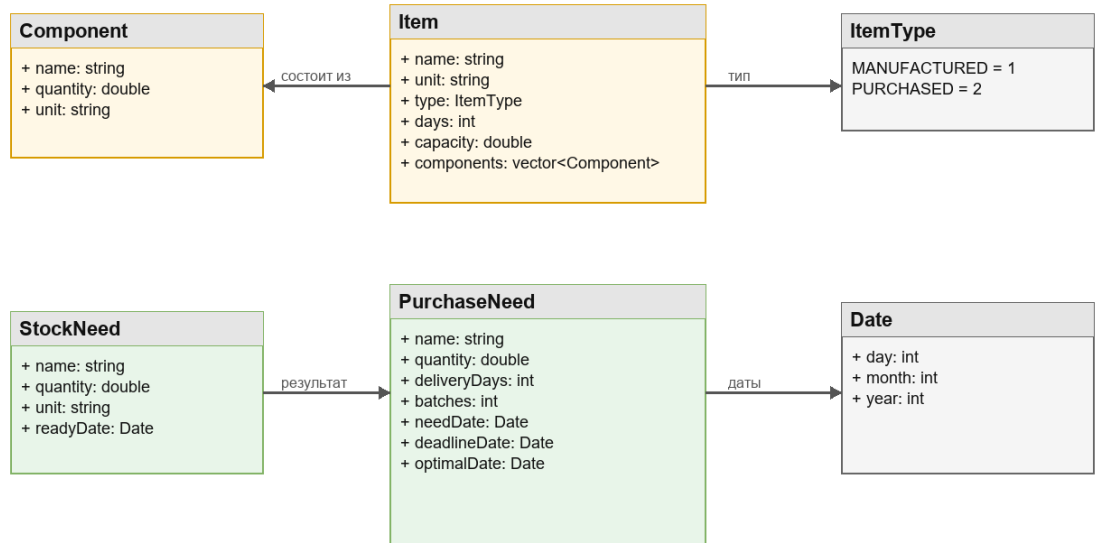


Рисунок 2 — Диаграмма структур данных

На второй диаграмме показаны структуры, описывающие предметную область. **Item** хранит основные параметры изделия: название, единицу измерения, тип, длительность цикла, ограничение вместимости и список компонентов. **Component** содержит название дочернего элемента, количество на одну единицу родителя и единицу измерения.

Структуры **StockNeed** и **PurchaseNeed** используются для вывода результата. **StockNeed** показывает, что и к какой дате должно быть на складе. **PurchaseNeed** хранит информацию о закупаемом материале: нужное количество, срок поставки, количество партий, крайнюю дату заказа и оптимальную дату заказа.

Расчетный модуль и зависимости

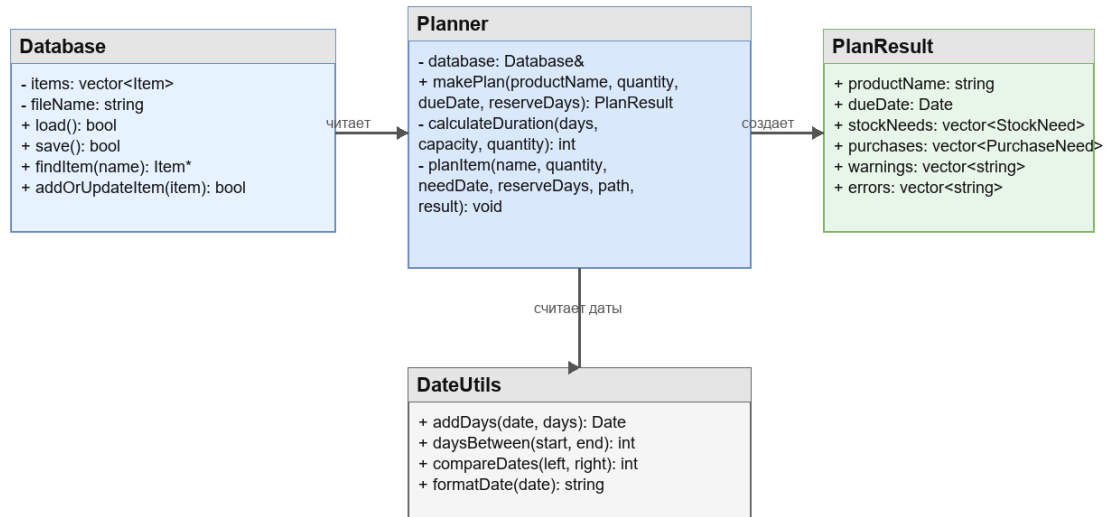


Рисунок 3 — Диаграмма расчетного модуля

На третьей диаграмме уточнена работа расчетного модуля. Planner использует Database для поиска записей и DateUtils для операций с датами. Метод makePlan() является основным открытым методом класса: он принимает название готового продукта, количество, дату готовности и запас дней.

Внутренний метод planItem() рекурсивно обходит состав изделия. Если запись закупается, создается PurchaseNeed. Если запись изготавливается, рассчитывается срок производства, дата потребности компонентов и дальнейший расчет продолжается для каждого дочернего элемента.

5. Реализация программного средства

Программное средство реализовано в виде Qt-приложения. Главная точка входа находится в QtMain.cpp, где создается QApplication, задается русская локаль и отображается MainWindow.

В MainWindow.cpp реализованы создание интерфейса, заполнение дерева базы, обработка кнопок добавления, изменения и удаления записей, запуск расчета и вывод результата в таблицы.

В ItemDialog.cpp реализовано окно записи. Для удобства пользователя состав изделия отображается деревом. Если компонент закупается, в строке сразу доступны поля срока поставки и закупки одной партией, без необходимости отдельно заходить в запись.

В Database.cpp реализована файловая база. Перед сохранением данные очищаются от пустых названий и некорректных компонентов. При загрузке испорченные строки пропускаются, чтобы программа не завершалась аварийно.

В Planner.cpp реализована главная идея проекта. Длительность изготовления или поставки рассчитывается с учетом ограничения capacity. Если требуется 10 изделий, а параллельно можно изготовить только 5, программа считает два производственных цикла.

Основная формула расчета длительности имеет вид: количество циклов равно округлению вверх от $\text{quantity} / \text{capacity}$, а итоговая длительность равна $\text{cycles} * \text{days}$. Если ограничение capacity не задано или меньше нуля, используется один цикл.

Для закупаемых позиций программа рассчитывает дату крайнего заказа как дату потребности минус срок поставки. Оптимальная дата заказа получается вычитанием пользовательского запаса из крайней даты.

6. Основные задачи разработки и принятые решения

6.1 Организация файловой базы данных

Для хранения записей была выбрана простая файловая база database.txt. Каждое изделие сохраняется строкой ITEM, а каждый компонент строкой COMP. Такой формат легко проверять вручную и просто обрабатывать в C++.

При удалении записи программа также удаляет зависимости, чтобы в составе других изделий не оставались ссылки на несуществующий компонент.

6.2 Разделение интерфейса и бизнес-логики

Расчет сроков не размещен внутри интерфейса. Он вынесен в класс Planner. Благодаря этому расчет можно использовать как из Qt-версии, так и из консольной версии, оставленной для проверки алгоритма.

Такое решение повышает читаемость проекта и упрощает дальнейшую модернизацию, например подключение другой базы данных или изменение интерфейса.

6.3 Работа с вложенными компонентами

Важной задачей была поддержка многоуровневого состава изделия. Для этого компонент хранит имя дочерней записи, а сама запись с таким именем находится в базе. При расчете Planner переходит от изделия к его компонентам, затем к компонентам этих компонентов.

Для защиты от ошибок предусмотрена проверка циклической зависимости. Если изделие случайно окажется компонентом самого себя через цепочку других компонентов, программа добавит ошибку в результат расчета.

6.4 Проверка пользовательского ввода

В программе реализованы проверки пустых названий, неправильных дат, некорректных количеств и единиц измерения. Если единица измерения равна «шт», количество должно быть целым числом. Для других единиц допускаются дробные значения.

Поля Qt SpinBox и DateEdit ограничивают диапазоны значений, а дополнительные проверки в коде защищают данные перед сохранением.

6.5 Обработка нехватки времени

Если рассчитанная крайняя дата закупки оказывается раньше даты обращения, программа выводит предупреждение о том, что по срокам заказ выполнить невозможно. Дополнительно рассчитывается лучшая возможная дата готовности при заказе прямо сейчас и дата с учетом запаса.

Это делает результат полезным не только при успешном планировании, но и при анализе просроченных или слишком срочных заказов.

7. Тестирование программного средства

Тестирование проводилось путем проверки основных пользовательских сценариев в консольной и Qt-версии программы. Были проверены следующие действия:

- создание нового изделия, детали и закупаемого материала;
- добавление компонентов первого и вложенных уровней;
- редактирование ранее созданных записей;
- удаление записи вместе с зависимыми элементами;
- очистка всей базы;
- расчет изделия, состоящего из изготавливаемых и закупаемых компонентов;
- расчет при ограничении параллельного изготовления;
- проверка предупреждения при нехватке времени;
- ввод неправильной даты и проверка обработки ошибки;
- ввод дробного количества для единицы «шт» и проверка запрета такого значения.

В качестве контрольного примера использовалась структура изделия «Стул»: стул состоит из ножек, спинки и болтов; ножки требуют брус, спинка требует фанеру, а болты закупаются напрямую.

В результате тестирования было установлено, что программа корректно рассчитывает даты склада, объединяет одинаковые закупаемые материалы, выводит сроки закупки и сохраняет данные между запусками.

Заключение

В ходе выполнения работы было разработано программное средство «Оптимизация производственного закупа». Приложение позволяет вести базу изделий, деталей и материалов, описывать многоуровневый состав продукции и рассчитывать сроки закупки от требуемой даты готовности.

В проекте реализована модульная структура: модели данных, база, расчетный модуль, работа с датами и пользовательский интерфейс разделены по разным классам. Это упрощает сопровождение кода и дальнейшее развитие программы.

Использование Qt Widgets позволило создать простое графическое приложение с вкладками, таблицами, деревом базы и диалогом редактирования записей. Все основные действия выполняются через русскоязычный интерфейс.

Полученный результат может быть использован как основа для дальнейшего развития: подключения SQLite, формирования печатных отчетов, добавления ролей пользователей, учета календаря рабочих дней и автоматической оценки стоимости закупки.

Список использованных источников

1. Страуструп Б. Язык программирования C++. — Москва: Бином, 2020.
2. Бланшет Ж., Саммерфилд М. C++ GUI Programming with Qt 4. — Prentice Hall, 2008.
3. Qt Documentation [Электронный ресурс]. — Режим доступа: <https://doc.qt.io/> (дата обращения: 27.05.2026).
4. ГОСТ 7.32–2017. Отчет о научно-исследовательской работе. Структура и правила оформления.
5. Ларман К. Применение UML и шаблонов проектирования. — Москва: Вильямс, 2019.
6. Павловская Т. А. C/C++. Программирование на языке высокого уровня. — Санкт-Петербург: Питер, 2021.